



ASCII

Error Handling & Library



Error Handling

Error handling adalah proses mendeteksi, menangani, dan mengelola kesalahan atau error yang terjadi dalam program agar tidak menyebabkan program berhenti secara tiba-tiba. Tujuannya adalah memberikan respon yang tepat, seperti menampilkan pesan error yang jelas, mencoba solusi alternatif, atau memastikan program tetap berjalan dengan lancar.

Jenis-jenis Error

Jenis error yang terdapat dalam bahasa pemrograman C++ ada beragam, berikut adalah beberapa jenis error yang terdapat dalam bahasa pemrograman C++.

Jenis Error	Deskripsi	Contoh
Exception	Kelas dasar untuk semua exception atau error di C++	Kesalahan umum yang tidak ingin dispesifikasikan.
logic_error	Menangani error logika, seperti operasi yang tidak masuk akal.	Mencari akar kuadrat dari bilangan negatif
invalid_argument	Menangani argumen fungsi yang tidak valid.	Memasukkan argumen negatif ke fungsi yang membutuhkan bilangan positif.
runtime_error	Menangani error saat runtime yang tidak dapat diprediksi sebelumnya.	Membagi bilangan dengan nol dalam operasi
out_of_range	Mengakses elemen di luar batas array	Mengakses elemen indeks ke-5 dari array berukuran 3.
length_error	Menangani kesalahan ketika panjang array melebihi batas maksimal.	Menambahkan terlalu banyak elemen ke dalam array hingga melampaui batas kapasitasnya.



Exception Handling

Pada pemrograman, error handling sangat penting untuk memastikan program tetap berjalan walaupun terjadi kesalahan. Bahasa pemrograman C++ menyediakan mekanisme error handling menggunakan **try**, **throw**, dan **catch**. Dengan memahami fungsi dari ketiga elemen ini, Anda dapat menangani error secara efektif. Berikut penjelasan singkat mengenai masing-masing elemen:

1. Try

Blok untuk menempatkan kode yang berpotensi menimbulkan error.

2. Throw

Digunakan untuk melemparkan error saat error terjadi.

3. Catch

Blok untuk menangkap dan menangani error yang dilempar oleh throw.

Contoh 1:

```
#include <iostream>
using namespace std;
int main() {
    try {
        int a = 10;
        int b = 0;
        if (b == 0) {
            throw runtime_error("Error pembagian dengan 0");
        }
        int c = a / b;
        cout << "Hasil: " << c << endl;
    } catch (const runtime_error& e) {
        cout << "Error ditangkap: " << e.what() << endl;
    }
}
```



```
}  
return 0;  
}
```

- Program masuk ke dalam try dan mendeklarasikan dua variabel a dan b dengan nilai masing-masing 10 dan 0.
- Program memeriksa kondisi **if (b == 0)** dan menemukan bahwa kondisinya benar.
- Karena kondisinya terpenuhi, program menggunakan throw untuk melemparkan error berupa objek **runtime_error** dengan pesan "Error pembagian dengan 0".
- Error yang dilempar langsung ditangkap oleh catch yang menangani objek tipe **runtime_error**.
- Dalam catch, program memanggil metode what() dari objek **runtime_error** untuk menampilkan pesan error ke terminal.

Contoh 2:

```
#include <iostream>  
using namespace std;  
int main() {  
    try {  
        cout << "Masukkan bilangan positif: ";  
        int angka;  
        cin >> angka;  
        if (cin.fail()) {  
            throw invalid_argument("Input harus berupa angka!");  
        }  
        if (angka < 0) {  
            throw runtime_error("Bilangan harus bernilai positif!");  
        }  
        cout << "Anda memasukkan: " << angka << endl;  
    }
```



```
}  
  
catch (const invalid_argument& e) {  
    cout << "Error: " << e.what() << endl;  
}  
  
catch (const runtime_error& e) {  
    cout << "Error: " << e.what() << endl;  
}  
  
cout << "Program selesai." << endl;  
return 0;  
}
```

- Program meminta pengguna untuk memasukkan bilangan positif.
- Jika pengguna memasukkan bukan angka seperti huruf, program melempar error tipe `invalid_argument` dengan pesan "Input harus berupa angka!".
- Jika angka yang dimasukkan bernilai negatif, program melempar error tipe `runtime_error` dengan pesan "Bilangan harus bernilai positif!".
- Blok `catch` menangkap dan menampilkan pesan dari error yang dilempar.
- Jika input valid, program mencetak angka yang dimasukkan pengguna dan melanjutkan eksekusi dengan mencetak pesan "Program selesai."

Contoh 3:

```
#include <iostream>  
#include <stdexcept>  
using namespace std;  
  
void cekData(int* ptr) {  
    if (ptr == nullptr) {  
        throw runtime_error("Error: Pointer bernilai NULL!");  
    }  
}
```



```
        cout << "Nilai dalam pointer: " << *ptr << endl;
    }

int main() {
    try {
        int* dataKosong = nullptr;
        int dataAda = 100;

        cekData(&dataAda);

        cout << "Mencoba akses pointer kosong..." << endl;
        cekData(dataKosong);

    } catch (const runtime_error& e) {
        cout << "Exception ditangkap: " << e.what() << endl;
    }

    return 0;
}
```

- Fungsi **cekData** menerima pointer integer. Sebelum mengakses nilainya (dereference), fungsi memeriksa apakah **ptr == nullptr**.
- Jika **ptr** kosong, program melakukan **throw runtime_error** untuk mencegah crash.
- Di **main**, blok **try** menjalankan dua skenario. Skenario kedua mengirim **dataKosong** yang memicu error.
- Blok **catch** menangkap error tersebut dan menampilkan pesan peringatan dengan aman.



Debugging

Debugging adalah proses mencari penyebab kesalahan. teknik paling dasar yang bisa digunakan adalah **Print Debugging**. Teknik ini menempatkan **cout** di titik-titik krusial untuk melacak nilai variabel dan alur eksekusi program.

Contoh:

```
#include <iostream>
using namespace std;

int hitungFaktorial(int n) {
    int hasil = 1;
    cout << "[DEBUG] Memulai perhitungan untuk n = " << n << endl; //
    Debugging

    for (int i = 1; i ≤ n; i++) {
        hasil = hasil * i;
        // Melacak perubahan nilai 'hasil' di setiap perulangan
        cout << "[DEBUG] Iterasi ke-" << i << ", hasil sementara: "
    << hasil << endl;
    }
    return hasil;
}

int main() {
    int angka = 5;
    cout << "Program Faktorial" << endl;
```



```
int hasilAkhir = hitungFaktorial(angka);

cout << "Hasil Akhir: " << hasilAkhir << endl;
return 0;
}
```

Contoh 2:

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream> // Untuk menggabungkan string

using namespace std;

void laporError(const string& pesan, const char* file, int baris,
const char* fungsi) {
    stringstream ss;
    ss << "\n[CRITICAL ERROR DETECTED!]\n";
    ss << "-----\n";
    ss << "Pesan : " << pesan << endl;
    ss << "File : " << file << endl;
    ss << "Fungsi : " << fungsi << "()" << endl;
    ss << "Baris : " << baris << endl;
    ss << "-----\n";
}
```




```
        throw runtime_error(ss.str());
    }

#define ASSERT(kondisi, pesan) \
    if (!(kondisi)) { \
        laporError(pesan, __FILE__, __LINE__, __func__); \
    }

void prosesData(int index, int pembagi) {
    cout << "Sedang memproses data..." << endl;

    ASSERT(pembagi != 0, "Dilarang membagi dengan nol!");

    ASSERT(index <= 10, "Akses index melebihi batas array (Out of Range)!");

    int hasil = 100 / pembagi;
    cout << "Hasil proses: " << hasil << endl;
}

int main() {
    cout << "≡ PROGRAM PENDETEKSI KESALAHAN OTOMATIS ≡" << endl;

    try {
        // Skenario 1: Normal
        prosesData(5, 2);
    }
```



```
// Skenario 2: Error (Coba ubah parameter ini untuk mengetes)
cout << "\nMencoba skenario berbahaya..." << endl;
prosesData(15, 5);

} catch (const exception& e) {
    cout << e.what() << endl;
}

return 0;
}
```



Library

Library (pustaka) adalah kumpulan kode program yang telah dikompilasi dan dapat digunakan kembali untuk mempercepat proses pengembangan perangkat lunak. Dengan menggunakan library, programmer tidak perlu menulis ulang kode untuk fungsi-fungsi umum. Dalam C++, library dibagi menjadi dua jenis utama:

1. **Internal Library (Standard Library):** Library bawaan yang sudah disediakan oleh compiler C++. Contoh: `<iostream>`, `<cmath>`, `<vector>`, `<string>`.
2. **External / User-Defined Library:** Library yang dibuat oleh pihak ketiga atau dibuat sendiri oleh programmer untuk kebutuhan spesifik. Library ini biasanya disimpan dalam file berekstensi `.h` (header) atau `.cpp`.

Pemuatan library dilakukan menggunakan *preprocessor directive* `#include`.

- Menggunakan tanda kurung siku `< >` untuk library standar (misal: `#include <iostream>`).
- Menggunakan tanda kutip `" "` untuk library eksternal atau buatan sendiri (misal: `#include "myheader.h"`).

Contoh 1: Internal Library (cmath & vector) Library standar menyediakan fungsi matematika kompleks dan struktur data dinamis.

```
#include <iostream>
#include <cmath>    // Library untuk operasi matematika (pow, sqrt)
#include <vector>   // Library untuk array dinamis
#include <algorithm> // Library untuk algoritma (sort)

using namespace std;

int main() {
    double angka = 25.0;
```



```
double akar = sqrt(angka);
double pangkat = pow(angka, 2);

cout << "Akar dari " << angka << " adalah: " << akar << endl;
cout << "Pangkat dua dari " << angka << " adalah: " << pangkat <<
endl;

vector<int> data = {50, 10, 30, 20, 40};
sort(data.begin(), data.end());

cout << "Data terurut: ";
for (int n : data) {
    cout << n << " ";
}
cout << endl;

return 0;
}
```

Program ini menyertakan library **<cmath>** untuk menghitung akar kuadrat (**sqrt**) dan perpangkatan (**pow**). Program juga menggunakan **<vector>** sebagai pengganti array statis, yang memungkinkan ukuran array berubah secara dinamis. Fungsi **sort()** dipanggil dari library **<algorithm>** untuk mengurutkan data secara otomatis tanpa perlu membuat algoritma sorting manual (seperti bubble sort).

Contoh 2: External Library Untuk menggunakan library luar atau buatan sendiri, kita memisahkan deklarasi fungsi ke dalam file header (.h).

File 1: hitung.h

```
#ifndef HITUNG_H
#define HITUNG_H

int tambah(int a, int b) {
```



```
    return a + b;
}

int kali(int a, int b) {
    return a * b;
}

#endif
```

File 2: main.cpp

```
#include <iostream>
#include "hitung.h"

using namespace std;

int main() {
    int x = 10, y = 5;

    cout << "Hasil Tambah: " << tambah(x, y) << endl;
    cout << "Hasil Kali : " << kali(x, y) << endl;

    return 0;
}
```

File **hitung.h** bertindak sebagai library eksternal sederhana yang berisi definisi fungsi matematika. **#ifndef**, **#define**, dan **#endif** disebut *Include Guards*, berfungsi mencegah error jika library dipanggil berulang kali. Pada **main.cpp**, pemanggilan library menggunakan tanda kutip **#include "hitung.h"**. Ini memberitahu compiler untuk mencari file tersebut di folder proyek yang sama, bukan di folder sistem C++. Hasilnya, fungsi **tambah()** dan **kali()** dapat langsung digunakan di program, utama seolah-olah fungsi tersebut ditulis di sana.



Studi Kasus

1. Buatlah sebuah ilustrasi yang menggambarkan konsep error handling atau library
2. Buatlah sebuah program C++ untuk mencatat nilai mahasiswa. Program harus menggunakan vector (Internal Library) untuk menyimpan data nilai secara dinamis. Program harus mampu menangani kesalahan input dari pengguna agar tidak *crash*.